

```
#!/usr/bin/env python3
"""بوت تليغرام لبيع خدمات السوشيال ميديا"""

import os
import logging
from telegram import Update,
InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    Application, CommandHandler,
    CallbackQueryHandler,
    MessageHandler, filters, ContextTypes,
    ConversationHandler
)

from config import BOT_TOKEN, ADMIN_ID,
MASTERCARD_NUMBER, ASIACELL_NUMBER,
SUPPORT_USERNAME, MIN_DEPOSIT
import database as db
import smm_api as api
from services_catalog import PLATFORMS,
find_service, calc_price

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

(WAIT_LINK, WAIT_QUANTITY,
WAIT_DEPOSIT_AMOUNT, WAIT_DEPOSIT_PROOF,
WAIT_BROADCAST) = range(5)

def back_btn(data):
```

```

        return
    InlineKeyboardMarkup([[InlineKeyboardButton
    ("🔙 رجوع", callback_data=data)])

    async def main_menu(update: Update,
    context: ContextTypes.DEFAULT_TYPE):
        user = update.effective_user
        db.register_user(user.id, user.username
    or "", user.full_name)
        bal = db.get_balance(user.id)
        keyboard = [
            [InlineKeyboardButton("🛒 الخدمات",
    callback_data="platforms"),
            InlineKeyboardButton("💰 رصيدي",
    callback_data="balance")],
            [InlineKeyboardButton("📦 طلباتي",
    callback_data="my_orders"),
            InlineKeyboardButton("➕ شحن
    رصيد", callback_data="add_funds")],
            [InlineKeyboardButton("☎️ الدعم",
    callback_data="support")],
        ]
        text = (
            "🚀 *بوت خدمات السوشيال ميديا*\n\n"
            f"👋 أهلاً *{user.first_name}*\n"
            f"🏠 رصيدك الحالي: `{bal:.2f} $`\n\n"
            "اختر من القائمة أدناه:"
        )
        if update.message:
            await
            update.message.reply_text(text,
            reply_markup=InlineKeyboardMarkup(keyboard)
            , parse_mode="Markdown")
        else:

```

```

        await
update.callback_query.edit_message_text(text,
reply_markup=InlineKeyboardMarkup(keyboard)
, parse_mode="Markdown")

async def start(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    await main_menu(update, context)

async def show_balance(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    bal = db.get_balance(q.from_user.id)
    await q.edit_message_text(
        f"💰 *رصيـدك الحالي*\n\n`{bal:.4f}`
$`\n\nشحن رصيدك ➕ عبر زر «",
        reply_markup=back_btn("main_menu"),
        parse_mode="Markdown"
    )

async def show_platforms(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    keyboard = []
    row = []
    for key, plat in PLATFORMS.items():

row.append(InlineKeyboardButton(plat["name"
], callback_data=f"plat_{key}"))
        if len(row) == 2:
            keyboard.append(row); row = []

```

```

        if row:
            keyboard.append(row)

keyboard.append([InlineKeyboardButton("⬅️  
رجوع", callback_data="main_menu")])
await q.edit_message_text("🌐 *اختر  
المنصة:*",
reply_markup=InlineKeyboardMarkup(keyboard)
, parse_mode="Markdown")

async def show_services(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    platform_key = q.data.replace("plat_",
    "")
    plat = PLATFORMS.get(platform_key)
    if not plat:
        await q.answer("!منصة غير موجودة!",
show_alert=True); return
    context.user_data["platform"] =
platform_key
    keyboard = []
    for svc in plat["services"]:

keyboard.append([InlineKeyboardButton(f"
{svc['name']} - {svc['price_per_1k']}$/
1000", callback_data=f"svc_{svc['id']}")])

keyboard.append([InlineKeyboardButton("⬅️  
رجوع", callback_data="platforms")])
await q.edit_message_text(f"
{plat['name']} - *اختر الخدمة:*",
reply_markup=InlineKeyboardMarkup(keyboard)

```

```
, parse_mode="Markdown")

async def select_service(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    svc =
find_service(q.data.replace("svc_", ""))
    if not svc:
        await q.answer("!خدمة غير موجودة!",
show_alert=True); return
    context.user_data["service"] = svc
    await q.edit_message_text(
        f"✅ *{svc['name']}*\n\n💰 السعر:
`{svc['price_per_1k']}$` 1000 لكل\n"
        f"📊 الحد الأدنى: `{svc['min']}` | الحد
الأكصى: `{svc['max']}`\n\n🔗 *رابط* أرسل الصفحة:",

reply_markup=back_btn(f"plat_{context.user_
data.get('platform', 'instagram')}"),
parse_mode="Markdown"
    )
    return WAIT_LINK

async def receive_link(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    link = update.message.text.strip()
    if not link.startswith("http"):
        await update.message.reply_text("❌
http الرابط غير صحيح، أرسل رابطاً يبدأ بـ
return WAIT_LINK
    context.user_data["link"] = link
    svc = context.user_data["service"]
```

```

        await update.message.reply_text(
            f"❗️ *الكمية المطلوبة* إلى \n(من\n
            `{svc['min']}` إلى `{svc['max']}`)",
            parse_mode="Markdown"
        )
        return WAIT_QUANTITY

    async def receive_quantity(update: Update,
                                context: ContextTypes.DEFAULT_TYPE):
        try:
            qty =
            int(update.message.text.strip())
        except ValueError:
            await update.message.reply_text("❌ أرسل رقماً صحيحاً فقط")
            return WAIT_QUANTITY
        svc = context.user_data["service"]
        link = context.user_data["link"]
        user_id = update.effective_user.id
        if qty < svc["min"] or qty >
        svc["max"]:
            await
            update.message.reply_text(f"❌ الكمية خارج\n
            من \n!النطاق\n
            `{svc['min']}` إلى\n
            `{svc['max']}`", parse_mode="Markdown")
            return WAIT_QUANTITY
        price = calc_price(svc, qty)
        balance = db.get_balance(user_id)
        context.user_data["quantity"] = qty
        context.user_data["price"] = price
        keyboard = InlineKeyboardMarkup([
            InlineKeyboardButton("✅ تأكيد الطلب",
                                callback_data="confirm_order"),
            InlineKeyboardButton("❌ إلغاء",
                                callback_data="main_menu")
        ])

```

```

    ])
    await update.message.reply_text(
        f"📋 *ملخص الطلب:* \n\n🔧 الخدمة:
        `{svc['name']}` \n🔗 الرابط: `{link}` \n"
        f"📦 الكمية: `{qty}` \n💰 التكلفة:
        `{price:.4f}` $ \n🏠 رصيدك: `{balance:.4f}`
        $ \n\n"
        + ("✅ رصيدك كافٍ" if balance >=
price else "❌ رصيدك غير كافٍ - اشحن أولاً"),
        reply_markup=keyboard if balance >=
price else back_btn("main_menu"),
        parse_mode="Markdown"
    )
    return ConversationHandler.END

async def confirm_order(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    user_id = q.from_user.id
    svc = context.user_data.get("service")
    link = context.user_data.get("link")
    qty = context.user_data.get("quantity")
    price = context.user_data.get("price")
    if not all([svc, link, qty, price]):
        await q.edit_message_text("❌ حدث خطأ، حاول مجدداً")
        return
    db.update_balance(user_id, -price)
    order_id = db.create_order(user_id,
svc["id"], svc["name"], link, qty, price)
    result = api.create_order(svc["id"],
link, qty)
    smm_id = str(result.get("order",
"manual"))

```

```

db.update_order_smm_id(order_id,
smm_id, "processing")
    try:
        await
q.get_bot().send_message(ADMIN_ID,
    f"🆕 *طلب جديد #{order_id}* 🧑‍💻
المستخدم: `{user_id}` 🛠️ الخدمة:
{svc['name']}`\n"
    f"🔗 الرابط: {link}\n📦 الكمية:
{qty}\n💰 السعر: {price:.4f}$\n📡 SMM ID:
`{smm_id}`",
        parse_mode="Markdown")
    except Exception:
        pass
    await q.edit_message_text(
        f"✅ *تم إرسال طلبك بنجاح!* 📄 رقم
الطلب: `{order_id}` 📡 SMM:
`{smm_id}` 🕒 سيبدأ التنفيذ خلال دقائق",
        reply_markup=back_btn("main_menu"),
        parse_mode="Markdown"
    )

async def my_orders(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    orders =
db.get_user_orders(q.from_user.id)
    if not orders:
        await q.edit_message_text("لا يوجد 📦
لديك طلبات حتى الآن.",
        reply_markup=back_btn("main_menu")); return
    STATUS_EMOJI = {"pending": "🕒",
"processing": "🔄", "completed": "✅",

```



```

"partial": "⚠️", "cancelled": "❌"}
    lines = ["📦 *آخر طلباتك:*\n"]
    for o in orders:
        oid, sname, link, qty, price,
status, created = o
        lines.append(f"
{STATUS_EMOJI.get(status, '?')} *#{oid}* -
{sname}\n 📦 {qty} | 💰 {price:.4f}$ |
{status}\n 🕒 {created[:10]}\n")
        await
q.edit_message_text("\n".join(lines),
reply_markup=back_btn("main_menu"),
parse_mode="Markdown")

async def add_funds(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    keyboard = InlineKeyboardMarkup([
        [InlineKeyboardButton("🇲🇦 ماستر
كارد", callback_data="pay_mastercard")],
        [InlineKeyboardButton("🇸🇩 آسياسيل
كاش", callback_data="pay_asiacell")],
        [InlineKeyboardButton("⬅️ رجوع",
callback_data="main_menu")],
    ])
    await q.edit_message_text("✚️ *شحن
اختار طريقة الدفع*\n\nالرصيد",
reply_markup=keyboard,
parse_mode="Markdown")

async def pay_mastercard(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query

```

```

        await q.answer()
        context.user_data["pay_method"] =
"Mastercard"
        await q.edit_message_text(
            f"💳 *الدفع عبر ماستر كارد*\n\nرقم
البطاقة:\n`{MASTERCARD_NUMBER}`\n\n⚠️ الحد
الأدنى: `{MIN_DEPOSIT}`$\n\nأرسل بعد التحويل
المبلغ* الذي دفعته",
            reply_markup=back_btn("add_funds"),
            parse_mode="Markdown"
        )
        return WAIT_DEPOSIT_AMOUNT

async def pay_asiacell(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    context.user_data["pay_method"] =
"Asiacell"
    await q.edit_message_text(
        f"📞 *الدفع عبر آسياسيل كاش*\n\nرقم
الحد الأدنى: \n`{ASIACELL_NUMBER}`\n\n⚠️
الحد الأدنى: `{MIN_DEPOSIT}`$\n\nأرسل
المبلغ* الذي دفعته:",
        reply_markup=back_btn("add_funds"),
        parse_mode="Markdown"
    )
    return WAIT_DEPOSIT_AMOUNT

async def receive_deposit_amount(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    try:
        amount =

```

```

float(update.message.text.strip())
except ValueError:
    await update.message.reply_text("❌
(10 : مثال : 10)"); return
WAIT_DEPOSIT_AMOUNT
if amount < MIN_DEPOSIT:
    await
update.message.reply_text(f"❌ الحد الأدنى
{MIN_DEPOSIT}$"); return
WAIT_DEPOSIT_AMOUNT
context.user_data["deposit_amount"] =
amount
await update.message.reply_text(f"✅
الآن أرسل *صورة إيصال* 📷 *{amount}$*\n\n
التحويل:", parse_mode="Markdown")
return WAIT_DEPOSIT_PROOF

async def receive_deposit_proof(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    amount =
context.user_data.get("deposit_amount", 0)
    method =
context.user_data.get("pay_method",
"unknown")
    proof =
update.message.photo[-1].file_id if
update.message.photo else
(update.message.text or "no_proof")
    dep_id = db.create_deposit(user_id,
amount, method, proof)
    keyboard = InlineKeyboardMarkup([[
        InlineKeyboardButton("✅ قبول",

```

```

callback_data=f"dep_approve_{dep_id}"),
    InlineKeyboardButton("❌ رفض",
callback_data=f"dep_reject_{dep_id}")
    ])
    msg = f"💰 *طلب شحن جديد* #{dep_id}*\n\n
👤 المستخدم: `{user_id}`\n📄 المبلغ:
`{amount}$`\n🛡️ الطريقة: {method}"
    try:
        if update.message.photo:
            await
update.get_bot().send_photo(ADMIN_ID,
photo=proof, caption=msg,
reply_markup=keyboard,
parse_mode="Markdown")
        else:
            await
update.get_bot().send_message(ADMIN_ID, msg
+ f"\n📄 الإيصال: {proof}",
reply_markup=keyboard,
parse_mode="Markdown")
    except Exception as e:
        logger.error(f"خطأ: {e}")
        await update.message.reply_text(
            f"✅ *تم استلام طلب الشحن* #
{dep_id}*\n\n⌚ سيتم مراجعته وإضافة الرصيد خلال
{SUPPORT_USERNAME}.\nدقائق",
            parse_mode="Markdown"
        )
    return ConversationHandler.END

async def admin_approve_deposit(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    q = update.callback_query

```

```

        if q.from_user.id != ADMIN_ID:
            await q.answer("❌ غير مصرح!",
show_alert=True); return
            dep_id = int(q.data.split("_")[-1])
            user_id, amount =
db.approve_deposit(dep_id)
            if user_id:
                await q.answer("✅ تمت الموافقة")
                try:
                    await
q.get_bot().send_message(user_id,
                        f"🎉 *تم شحن رصيدك!**\n\n📊
رصيدك الجديد: `{amount}$`\n📊 المبلغ المضاف:
`{db.get_balance(user_id):.4f}$`",
                        parse_mode="Markdown")
                except Exception:
                    pass
            else:
                await q.answer("❌ خطأ في قاعدة
البيانات", show_alert=True)

async def admin_reject_deposit(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    if q.from_user.id != ADMIN_ID:
        await q.answer("❌ غير مصرح!",
show_alert=True); return
        db.reject_deposit(int(q.data.split("_")
[-1]))
        await q.answer("❌ تم الرفض")

async def support(update: Update, context:
ContextTypes.DEFAULT_TYPE):

```

```

q = update.callback_query
await q.answer()
await q.edit_message_text(
    f"📞 *تواصل معنا*\n\nمعا: {SUPPORT_USERNAME}\n\n24/7 ساعات العمل: 🕒",
    reply_markup=back_btn("main_menu"),
    parse_mode="Markdown"
)

async def admin_stats(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    if update.effective_user.id !=
ADMIN_ID: return
    users = db.get_all_users()
    await update.message.reply_text(
        f"📊 *إحصائيات البوت*\n\n👥 المستخدمين: `{len(users)}`\n\n💰 إجمالي الأرصدة: `{sum(u[3]
for u in users):.4f}`$",
        parse_mode="Markdown"
    )

async def admin_add_balance(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    if update.effective_user.id !=
ADMIN_ID: return
    try:
        _, uid, amt =
update.message.text.split()
        db.update_balance(int(uid),
float(amt))
        await
update.message.reply_text(f"✅ تمت إضافة {amt}$ لـ {uid}")
    except Exception:

```

```
        await update.message.reply_text("❌  
الاستخدام: /addbal <user_id> <amount>")
```

```
async def callback_router(update: Update,  
context: ContextTypes.DEFAULT_TYPE):  
    q = update.callback_query  
    data = q.data  
    if data == "main_menu":        await  
main_menu(update, context)  
    elif data == "balance":        await  
show_balance(update, context)  
    elif data == "platforms":      await  
show_platforms(update, context)  
    elif data == "my_orders":      await  
my_orders(update, context)  
    elif data == "add_funds":      await  
add_funds(update, context)  
    elif data == "support":        await  
support(update, context)  
    elif data == "confirm_order":  await  
confirm_order(update, context)  
    elif data.startswith("plat_"):  
await show_services(update, context)  
    elif data.startswith("dep_approve_"):  
await admin_approve_deposit(update,  
context)  
    elif data.startswith("dep_reject_"):  
await admin_reject_deposit(update, context)  
    else: await q.answer()  
  
def main():  
    db.init_db()  
    app =  
Application.builder().token(BOT_TOKEN).buil
```

```

d()

    order_conv = ConversationHandler(
        entry_points=
[CallbackQueryHandler(select_service,
pattern=r"^svc_")],
        states={
            WAIT_LINK:
[MessageHandler(filters.TEXT &
~filters.COMMAND, receive_link)],
            WAIT_QUANTITY:
[MessageHandler(filters.TEXT &
~filters.COMMAND, receive_quantity)],
        },
        fallbacks=[CommandHandler("start",
start)],
        per_message=False
    )

    deposit_conv = ConversationHandler(
        entry_points=[

CallbackQueryHandler(pay_mastercard,
pattern="^pay_mastercard$"),

CallbackQueryHandler(pay_asiacell,
pattern="^pay_asiacell$"),
        ],
        states={
            WAIT_DEPOSIT_AMOUNT:
[MessageHandler(filters.TEXT &
~filters.COMMAND, receive_deposit_amount)],
            WAIT_DEPOSIT_PROOF:
[MessageHandler(filters.PHOTO |

```



```
filters.TEXT, receive_deposit_proof)],
    },
    fallbacks=[CommandHandler("start",
start)],
    per_message=False
)

app.add_handler(CommandHandler("start",
start))

app.add_handler(CommandHandler("stats",
admin_stats))

app.add_handler(CommandHandler("addbal",
admin_add_balance))
    app.add_handler(order_conv)
    app.add_handler(deposit_conv)

app.add_handler(CallbackQueryHandler(callba
ck_router))

    logger.info("✅ البوت يعمل...")

app.run_polling(drop_pending_updates=True)

if __name__ == "__main__":
    main()
```